

Multi-Fidelity Bayesian Optimization for NMPC Tuning: A Documentation and Analysis Report

Codebase walkthrough of MFBO_NMPC

May 2026

Abstract

This report documents the simulation, optimization, and analysis pipeline of the MFBO_NMPC project. The codebase implements a runtime-aware multi-fidelity Bayesian optimization (MFBO) workflow for bi-objective nonlinear model predictive control (NMPC) tuning on a reduced bioprocess dilution plant. We describe the plant model, the NMPC formulation with an LQR-derived terminal cost, the multi-fidelity closed-loop evaluation policy, the surrogate model used to extrapolate truncated-horizon costs, and the Bayesian-optimization loop that drives the search. We then walk through every root-level script whose name starts with `main` and every script under `Result analysis/`, identifying what each one accomplishes and which figures it generates. We provide per-figure discussion of the published artefacts and close with an overall assessment of the optimization behaviour, the surrogate’s modelling regime, and the practical implications for controller deployment.

1 Introduction

NMPC tuning is simulation-expensive: each candidate parameter vector requires a closed-loop simulation that solves a nonlinear program (NLP) at every sampling instant. The cost of such a simulation depends strongly on tuning choices — longer horizons, tighter weights, harder constraint regimes — and the objective landscape is non-smooth in a way that defeats gradient-based optimization. The MFBO_NMPC project addresses this problem with three coupled mechanisms: *(i)* a fidelity variable that truncates the closed-loop simulation horizon to reduce per-evaluation cost; *(ii)* a polynomial fraction surrogate that extrapolates truncated cost to a full-horizon estimate; and *(iii)* a runtime-aware Bayesian-optimization acquisition that explicitly penalizes expensive candidates while still pursuing hypervolume improvement on a bi-objective Pareto front.

The remainder of this document is organized as follows. Section 2 formalizes the plant, controller, surrogate, and optimization loop. Section 3 catalogues the root-level driver scripts. Section 4 covers the analysis scripts and the figures they produce. Section 5 provides per-figure discussion of the published artefacts. Section 6 synthesizes the cross-cutting observations and Section 7 concludes.

2 Methods

2.1 Plant model

The plant is a three-state reduced bioprocess dilution model with states $x = [V, X, S]^\top$ (volume, biomass, substrate) and inputs $u = [F_{\text{in},1}, F_{\text{in},2}, F_{\text{out}}]^\top$, governed by

$$\dot{V} = F_{\text{in},1} + F_{\text{in},2} - F_{\text{out}}, \quad (1)$$

$$\dot{X} = \left(-\frac{F_{\text{in},1} + F_{\text{in},2}}{V} + \mu(S) - k_d \right) X, \quad (2)$$

$$\dot{S} = \frac{(S_{\text{in}} - S)F_{\text{in},1} - SF_{\text{in},2}}{V} - \mu(S) X Y_{XS}^{-1}, \quad (3)$$

with Monod kinetics $\mu(S) = \mu_{\max} S / (K_s + S)$ and parameters $S_{\text{in}} = 70 \text{ g/L}$, $\mu_{\max} = 0.273 \text{ h}^{-1}$, $K_s = 0.007 \text{ g/L}$, $k_d = 0.006 \text{ h}^{-1}$, $Y_{XS} = 0.42042$. State and input bounds are $V \in [0.5, 2]$, $X \in [0, 50]$, $S \in [-0.1, 20]$, $u_i \in [0, 0.4]$, $i = 1, 2, 3$. Plant simulation uses `ode45` with non-negativity enforcement on X and S .

2.2 NMPC formulation with LQR-derived terminal cost

At each control step k , NMPC solves

$$\begin{aligned} \min_{\{u_{j|k}\}_{j=0}^{N_c-1}} \quad & \sum_{j=0}^{N_p-1} \|x_{j|k} - x_{sp}\|_Q^2 + \sum_{j=0}^{N_c-1} \|u_{j|k} - u_{sp}\|_{R_u}^2 \\ & + \sum_{j=0}^{N_c-1} \|\Delta u_{j|k}\|_{R_{\Delta u}}^2 + \left\| \begin{bmatrix} x_{N_p|k} - x_{sp} \\ u_{N_c-1|k} - u_{sp} \end{bmatrix} \right\|_P^2 \\ \text{s.t.} \quad & x_{0|k} = x_k, \quad x_{j+1|k} = f_{\text{plant}}(x_{j|k}, u_{j|k}), \\ & x_{j|k} \in \mathcal{X}, \quad u_{j|k} \in \mathcal{U}, \\ & u_{j|k} = u_{N_c-1|k}, \quad j = N_c, \dots, N_p - 1, \end{aligned} \quad (4)$$

solved with `fmincon-SQP` on scaled decision variables. Weights are parameterized in base-10 log-scale,

$$Q = \text{diag}(10^{q_1}, 10^{q_2}, 10^{q_3}), \quad R_u = \text{diag}(10^{r_{u,1}}, 10^{r_{u,2}}, 10^{r_{u,3}}), \quad R_{\Delta u} = \text{diag}(10^{r_{\Delta u,1}}, 10^{r_{\Delta u,2}}, 10^{r_{\Delta u,3}}). \quad (5)$$

Horizons are encoded as $N_c = \theta_m + 1$ and $N_p = \theta_p + N_c$ with integer variables $\theta_p \in [0, 15]$, $\theta_m \in [0, 7]$.

Free log-weight exponents are bounded in $[-3, 3]$, so the corresponding diagonal entries range from 10^{-3} to 10^3 .

Case 1 All variables are free.

Case 2 The first state-weight exponent is fixed at $q_1 = 0$, hence $Q_{11} = 1$. $r_u = -1000$ so R_u underflows to zero.

Terminal cost from an offline LQR design. The terminal cost matrix P is built offline. Linearizing the plant about its steady-state target yields (A, B) , which are discretized in incremental form to (A_i, B_i) at sample time $T_s = 1/60 \text{ h}$. A discrete LQR gain K is computed via `dlqr` using design weights $(Q_{\text{lqr}}, R_{u,\text{lqr}}, R_{\Delta u,\text{lqr}})$ whose log-exponents are obtained by an offline outer optimization (`TerminalCost/LQR_tuning.m`) that minimizes the closed-loop integrated squared error plus regularization terms over two representative initial conditions, $x_{0,\text{lqr}}^{(1)} = [1.1, 5, 1]^\top$ (off-volume, low biomass, low substrate; tests start-up tracking) and $x_{0,\text{lqr}}^{(2)} = [1, X_{sp,\text{lqr}} + 2, 0]^\top$ with $X_{sp,\text{lqr}} = 15$ (overshoot configuration that tests regulatory recovery from above the biomass setpoint). The resulting LQR tuning vector $\eta_{\text{lqr}} = [-1.998, 0.0003, 1.485, 0.527, -0.974, 0.043, 0.107, -0.118]$ is hard-coded as the design weights used for terminal-cost construction.

Because the NMPC stage weights $(Q, R_u, R_{\Delta u})$ generally differ from the LQR design weights, the terminal cost needs to be computed from the LQR gain matrix K under the new weights. Following standard infinite-horizon-evaluation theory (`TerminalCost/HowToCompute.m`), P is constructed by solving the discrete Lyapunov equation

$$P = A_{\text{cl}}^\top P A_{\text{cl}} + \bar{Q}, \quad \bar{Q} = S_x^\top Q S_x + (S_u - K)^\top R_u (S_u - K) + K^\top R_{\Delta u} K, \quad (6)$$

where A_{cl} is the LQR closed-loop matrix and S_x, S_u are state-/input-selection matrices from the augmented incremental formulation. The function `construct_P` (in `TerminalCost/LQR_functions/`) implements (6) and is called fresh for every candidate θ because $(Q, R_u, R_{\Delta u})$ vary across the search.

2.3 Multi-fidelity closed-loop evaluation

Terminology. Throughout this report we distinguish two orthogonal notions that are easily conflated:

- *Case 1 / Case 2* refer to the two independent BO tuning runs that differ in their input-penalty R_u configuration. Specifically, *Case 1* pins the three r_u exponents to -1000 in optimization space (so that $R_u = \text{diag}(10^{r_{u,1}}, 10^{r_{u,2}}, 10^{r_{u,3}})$ underflows to the zero matrix and the input penalty is effectively disabled), while *Case 2* releases those exponents and treats r_u as a free optimization variable in $[-3, 3]^3$. Wherever a figure label or numerical summary says “Case i ”, it means the i -th BO run.
- *Scenario* refers to a simulation setup (initial condition, setpoint trajectory, horizon, noise). Two *simulation scenarios* ($s = 1, 2$) share the same fixed setpoint ($V_{sp} = 1$, $X_{sp} = 20$) and 10 h horizon but use different initial conditions. The first, $x_0^{(1)} = [1.0, 10, 0]^\top$, represents a typical fermentation start: the volume is already at its setpoint, biomass is moderate, and substrate is depleted so that input flow is required for biomass to grow. The second, $x_0^{(2)} = [1.1, 25, 5]^\top$, is designed to keep $F_{\text{in},2}$ (u_2) from collapsing to zero while its associated R_u and $R_{\Delta u}$ increases to their upper bound; it tests regulatory action that must approach the setpoint from above and compensate biomass overshoot. Both scenarios are evaluated inside every BO query. A separate *validation scenario* is the 30 h three-step setpoint schedule $X_{sp} : 7 \rightarrow 13 \rightarrow 16$ used in Section 5 to test transferability.

Each candidate $\theta = [f, \theta_p, \theta_m, q, r_u, r_{\Delta u}] \in \mathbb{R}^{12}$ is evaluated by simulating both simulation scenarios $s \in \{1, 2\}$ over a fidelity-dependent horizon

$$t_f(f) = 10f \text{ h}, \quad N(f) = \lceil t_f(f)/T_s \rceil + 1, \quad f \in [0.01, 1]. \quad (7)$$

The truncated-horizon bi-objective is defined by summing across both simulation scenarios,

$$J_{\text{track}}^{\text{part}}(\theta, f) = \text{SSE}^{\text{part}}(\theta, f) = \sum_{s=1}^2 \sum_{k=0}^{N(f)-1} \|(x_{k,s} - x_{sp}) \odot [10, 1, 1]\|_2^2, \quad (8)$$

$$J_{\text{TV}}^{\text{part}}(\theta, f) = \text{SSdU}^{\text{part}}(\theta, f) = \sum_{s=1}^2 \sum_{k=1}^{N(f)-1} \|u_{k,s} - u_{k-1,s}\|_2^2, \quad (9)$$

The full-horizon estimates used by BO follow from a single Chebyshev fraction model per objective,

$$J_{\text{track}}(\theta, f) = \frac{J_{\text{track}}^{\text{part}}(\theta, f)}{\phi_{\text{SSE}}(f)}, \quad J_{\text{TV}}(\theta, f) = \frac{J_{\text{TV}}^{\text{part}}(\theta, f)}{\phi_{\text{SSdU}}(f)}, \quad (10)$$

with $\phi_i(f) = \text{clip}(\text{Cheb}_5(2f - 1; c_i), 0.01, 1)$ for $i \in \{\text{SSE}, \text{SSdU}\}$. The fixed Chebyshev coefficients (six values per objective) are fitted offline in `J_surrogate/surrogate.m` on normalized cumulative trajectories, achieving $R^2 = 0.9998$ for SSE and $R^2 = 0.9919$ for SSdU.

2.4 Bayesian-optimization setup

The optimization is cast as maximization of $y(\theta) = -[J_{\text{track}}(\theta), J_{\text{TV}}(\theta)]^\top$. Two Gaussian-process models are fit at every iteration: $\mathcal{M}_{\text{obj}} : \theta \mapsto y(\theta)$ and $\mathcal{M}_{\log t} : \theta \mapsto \log t(\theta)$, both using `SingleTaskGP` with input normalization and output standardization. The acquisition is a log-domain composition of `qLogNoisyExpectedHypervolumeImprovement` (`qLogNEHVI`) with a fidelity-bias term and a runtime penalty,

$$\log \alpha(\theta) = \log \text{qLogNEHVI}(\theta) + w_z \log(a_z(f) + \varepsilon) - w_t \gamma \log(\mathbb{E}[t(\theta)] + \varepsilon), \quad (11)$$

$$a_z(f) = \exp\left(-\frac{(1-f)^2}{2\ell_z^2}\right), \quad \mathbb{E}[t(\theta)] = \exp\left(\mu_{\log t}(\theta) + \frac{1}{2}\sigma_{\log t}^2(\theta)\right), \quad (12)$$

with $\ell_z = 0.25$, $\gamma = 1$, $w_z = w_t = 1$, $\varepsilon = 10^{-6}$, and $\mathbb{E}[t]$ lower-bounded by the best observed runtime. Reference points for the hypervolume calculation are updated from data as $r_j = y_{j,\min} - 0.2(y_{j,\max} - y_{j,\min})$. Each run uses $N_{\text{init}} = 20$ scrambled-Sobol design-of-experiment (DOE) points followed by 101 optimization iterations (121 evaluations total). Two runs are reported under identical settings to expose path dependence.

2.5 Software organization

The codebase is organized into two layers. Root-level **main*** scripts drive every kind of NMPC simulation needed (live MFBO loop, fixed Pareto replays, fixed-benchmark controllers, setpoint-schedule comparison). The **Result analysis/** folder transforms simulation artefacts into figures and tables for the manuscript. A single orchestrator, **Result analysis/run_analysis.m**, regenerates the entire figure set in one invocation.

3 Root-level driver scripts

3.1 main.py

The Python side of the MFBO loop. It builds a scrambled-Sobol DOE in optimization space, then iterates by fitting the two GPs, forming the qLogNEHVI acquisition wrapped in a custom **MFMOLogAcq** class that adds the fidelity bias $w_z \log a_z(f)$ and the runtime penalty $-w_t \gamma \log \mathbb{E}[t]$. Communication with MATLAB is via file I/O: θ is written to **inbox/theta.txt**, the worker writes new rows to **results/results.csv**, and Python logs each evaluation plus acquisition diagnostics to **results/opt_trace.csv**. Integer dimensions are rounded after acquisition. Three input-weight dimensions are pinned to -1000 so that 10^{r_u} underflows to zero, disabling the input penalty as a design choice.

3.2 main_BO.m

The MATLAB-side canonical evaluator. It initializes a **base** struct once (plant, setpoints, LQR data, fixed noise realization, Chebyshev coefficients) and runs in one of three modes: **external** (polls **inbox/theta.txt**), **doe** (sweeps a predefined matrix), or **single** (one hard-coded θ for debugging). The shared evaluator **simulate_nmpc** runs the closed loop on both simulation scenarios for $t_f = 10f$ h, then divides the truncated SSE/SSdU sums by $\phi_i(f)$ (floored at 0.01). Each θ produces one row in **results.csv** and one **.mat** payload.

3.3 main_NMPC_test_runs.m

Replays a hand-picked list of Pareto-optimal controllers at full fidelity ($f = 1$) and zero noise. Output trajectories are written to **results/test_run/<run>_full_f1_no_noise/**. Downstream consumers: **analyze_test_run_metrics.m**.

3.4 main_benchmark_fix.m

Runs a fixed reference NMPC ($m = 6$, $p = 61$, $Q = \text{diag}(10, 1, 1)$, $R_u = 0$, $R_{\Delta u} = \text{diag}(10, 10, 10)$, $P = 0$) at $f = 1$ both noise-free and with the same-noise realization used during BO. This is the baseline against which Pareto controllers are compared.

3.5 main_benchmark_reference_controller.m

Identical to **main_benchmark_fix.m** but retains the default input penalty $R_u = \text{diag}(2, 2, 1)$ from **NMPC_terminal**. Provides a controlled comparison isolating the effect of R_u .

3.6 main_final_fidelity.m

Replays all non-DOE Pareto-frontier controllers at $f = 1$ with the same noise realization as BO. This is the “refined” evaluation set used to validate whether BO’s surrogate-extrapolation ranking matches the true full-horizon ranking.

3.7 main_setpoint_schedule_controller_comparison.m

Simulates each Pareto-selected controller plus the benchmark on a 30 h setpoint-schedule scenario where the biomass target X_{sp} steps through $7 \rightarrow 13 \rightarrow 16$ at $t = 10, 20$ h. Non-benchmark controllers rebuild P at each setpoint change via the workflow in (6); the benchmark keeps $P = 0$. Evaluates transferability of the BO-tuned set to a regime never seen during optimization.

4 Analysis scripts

4.1 main_pareto.m

Post-processing hub. Loads `results.csv` from each run, enforces the policy that DOE iterations 1–20 are excluded from any Pareto/optimization conclusion, computes the non-dominated mask in (SSE, SSdU) space, summarises runtime by phase, and produces publication-ready figures plus numerical tables. Generates `pareto_samples_run1_run2.pdf`, `sse_vs_ssdu_side_by_side_z.pdf`, `runtime_vs_iteration_side_by_side.pdf`, `runtime_cumulative_run1_run2.pdf`, and `refined_frontier_ch`. Numerical outputs include `resultssandbox_runtime_and_params.txt` and `final_pareto_frontier_f1_noisy`.

4.2 resultssandbox.m

Predecessor of `main_pareto.m`. Produces the same first four PDFs and the same runtime summary, but omits the refined-frontier comparison and the noisy/noiseless metrics export. Useful as a lighter standalone entry point.

4.3 analyze_test_run_metrics.m

Computes settling-time, final-error, and IAE diagnostics from the noise-free full-horizon replays. Settling for state i is defined as the earliest time at which $|e_i|/|r_i| \leq 5\%$ for all subsequent samples; NaN indicates the final relative error exceeds tolerance. Exports the controller-by-scenario table consumed by `recover_boxplot_data.py`. Numerical outputs: `analyze_test_run_metrics_summary.txt`, `final_pareto_counts_by_case.txt`, and the boxplot CSVs.

4.4 recover_boxplot_data.py

Reads the boxplot CSVs and rebuilds the publication-quality figures `python_settling_time_boxplot.pdf`, `python_np_by_case_boxplot.pdf`, and `python_nc_by_case_boxplot.pdf`, using the Wong (2011) Nature Methods palette and a horizontal reference at $t = 8.09603$ h corresponding to the combined mean optimization-phase fidelity $z \cdot T$.

4.5 analyze_setpoint_schedule_metrics.m

Loads every `out_schedule_*.mat` from the validation scenario, computes per-scenario SSE/SSdU/IAE/settling totals (one entry per tuning-scenario initial condition), builds the 2-objective Pareto subset, and produces both trajectory plots and modified-versus-unmodified boxplots. Generates the manuscript-named PDFs `results/setpoint_tracking.pdf` and `results/setpoint_tracking_MV.pdf` alongside the `results/graphics/setpoint_schedule_*` variants. Numerical output: `setpoint_schedule_metrics_*`.

Selection of the deployment controller. The script identifies candidates programmatically and then commits to one by hand. First, it computes the 2-objective Pareto set on the schedule scenario using $(SSE_{\text{total}}, SSdU_{\text{total}})$. Second, it restricts to non-benchmark Pareto candidates, computes SSE_{total} relative to the benchmark, sorts ascending, and keeps the top three as `coloredControllerIds`. Third, one of those candidates is hard-coded as the deployment pick (`blackControllerId = "ts_20260211_1226"`). The documented selection rationale: against the benchmark, this controller achieves $16.6\times$ faster wall time (6.02% of the benchmark's), a $4.49\times$ smaller volume tracking error, an 8.87% lower SSdU, and biomass IAE within 2%, at a cost of 11.54% worse substrate tracking. Two commented-out alternatives (`ts_20260210_180826_modified` and `ts_20260210_151703_modified`) each beat the benchmark on subsets of these metrics but lose on others, typically trading substrate quality for volume quality. The selection is therefore a deliberate multi-criteria engineering pick rather than a single-objective optimum.

4.6 `plot_refined_frontier_change.m`

Directly compares the optimization Pareto frontier (after DOE exclusion) against the refined frontier obtained by re-evaluating the same controllers at $f = 1$ with the same noise. Generates `refined_frontier_change.pdf` and the textual report `results/txt results/refined_promoted_frontier_z`.

4.7 `exploring_u.m`

Post-hoc input smoothing experiment for the selected controller. Inputs are partitioned at nodes every ≈ 0.1 h. For each interval between consecutive nodes and each input channel, a linear program enforces the cumulative-sum constraint $\sum_i u_i^* = \sum_i u_i$ while minimizing $\sum_i |u_i^* - u_{i-1}^*|$. Generates `exploring_u_selected_controller_same_noise.pdf`.

4.8 `run_analysis.m`

Orchestrator. Runs `analyze_test_run_metrics.m`, then `recover_boxplot_data.py`, then `main_pareto.m`, then `J_surrogate/surrogate.m`, then `results/numerical results/compile_results_txt.m`, and finally copies every PDF in `results/graphical_results/` into `Overleaf--MFBO-NMPC/`.

5 Results: per-figure discussion

5.1 Pareto structure

Figure 1 shows a well-defined non-dominated frontier in both runs with a pronounced knee where further reduction in J_{track} requires disproportionately larger J_{TV} . The overlap between the two runs in the knee region is the principal evidence that this compromise is structurally robust for the tested plant and disturbance realization.

A second observation is that the two runs specialize on different segments of the front. Case 1 populates the low- J_{TV} tail (J_{TV} down to 0.026 but J_{track} up to 22,781), while Case 2 populates the low- J_{track} region (J_{track} as low as 12,221 with $J_{\text{TV}} \in [0.13, 0.39]$). Both runs concentrate at $N_c = 1$, but Case 1's Pareto subset frequently also picks $N_p = 1$ (5 of 9 candidates), whereas Case 2 picks $N_p = 1$ in only 4 of 10. When $N_c = N_p = 1$, the NMPC optimization horizon collapses to one stage cost plus the terminal P , so the resulting policy reduces to the LQR feedback law that built P applied to the nonlinear model at the current state — a smooth, conservative controller (low J_{TV}) that converges slowly (higher J_{track}). When $N_c = 1$ but $N_p > 1$, a single control move is committed but evaluated against an N_p -step prediction of free-state evolution under that move, closed off by the same terminal P . This makes the controller more anticipatory, lowering J_{track} at the cost of larger move-to-move corrections. The apparent “different degrees of freedom” between the two runs is therefore the same $N_c = 1$ regime exploring two different sub-modes, with BO's finite-budget path dependence pushing each run into a different sub-mode.

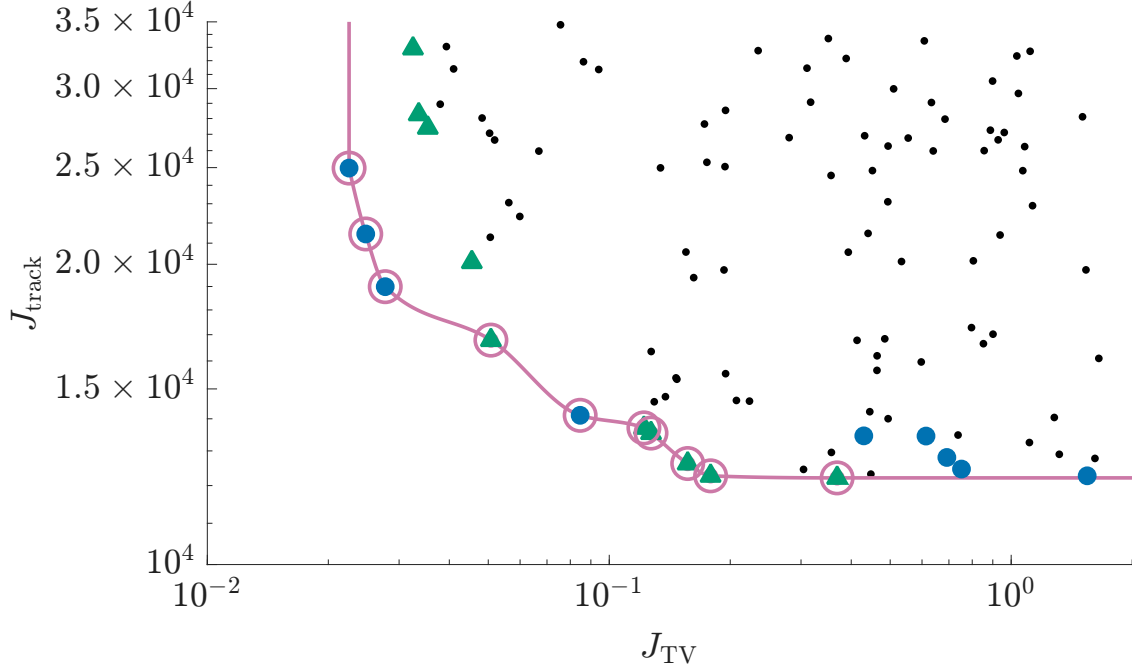


Figure 1: Evaluated samples in $(J_{TV}, J_{\text{track}})$ space and Pareto-optimal points across both BO runs. Each point is one BO evaluation aggregated across the two simulation scenarios. Small black filled dots are all evaluated optimization samples from both runs pooled together. Blue filled circles mark the within-case non-dominated points of Case 1 (R_u pinned to zero), and vermillion filled triangles mark the within-case non-dominated points of Case 2 (R_u as a free variable). Large open orange circles highlight the combined non-dominated frontier obtained when the two runs are merged into a single pool.

Benchmark comparison. The fixed-benchmark replay at $f = 1$ with the same noise realization used during BO, aggregated across both simulation scenarios, gives $\text{SSE} \approx 12,260$ with $\text{SSdU} \approx 3.52$. The top Case 2 Pareto controllers achieve essentially identical tracking ($\text{SSE} \approx 12,221\text{--}12,299$) with $\text{SSdU} \in [0.16, 0.39]$ — roughly 8 to $22\times$ less control variation. The benchmark therefore sits strictly inside the BO-found Pareto set on this plant: it is excessively aggressive on input movement without an offsetting tracking advantage.

5.2 Runtime behaviour

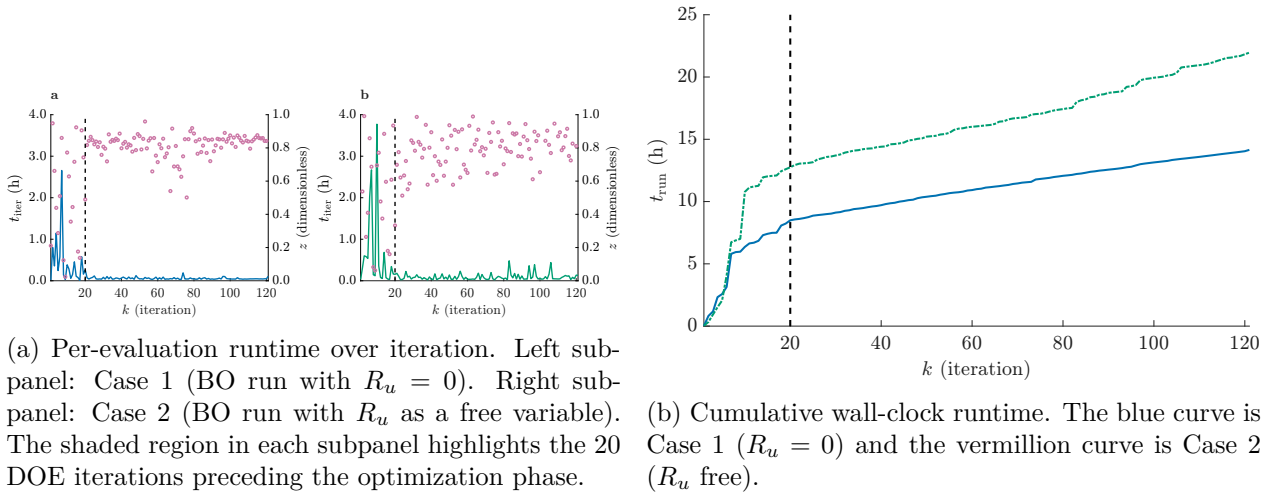


Figure 2: Runtime allocation across DOE and optimization phases for both BO runs.

Figure 2 confirms that DOE dominates computational cost in both runs. Case 1 spends 509.69 min in DOE (60.03%) and 339.36 min in optimization (39.97%), for a total of 849.05 min. Case 2 spends 765.23 min in DOE (58.08%) and 552.21 min in optimization (41.92%), for a total of 1,317.44 min. Combined across runs, 1,274.92 min of 2,166.49 min (58.85%) are spent in DOE. Average per-evaluation runtime drops from 31.87 min during DOE to 4.41 min during optimization, confirming that the runtime-aware acquisition shifts the search toward cheaper candidates after model initialization. Crucially, the mean optimization-phase fidelity remains high (0.819 in Case 1, 0.800 in Case 2, combined 0.810), so the runtime savings are not produced by collapsing to persistently low fidelity.

5.3 Fidelity-coloured objective scatter

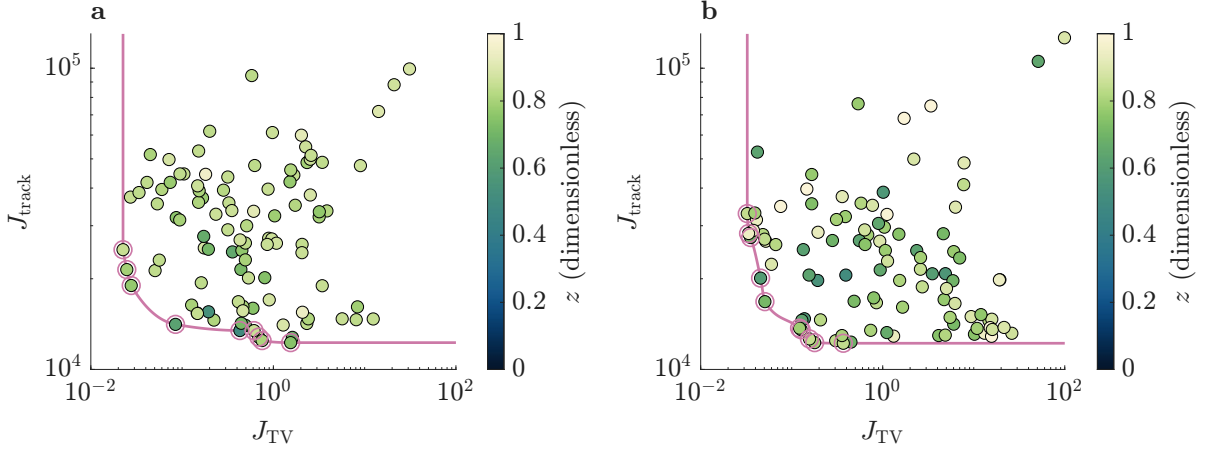
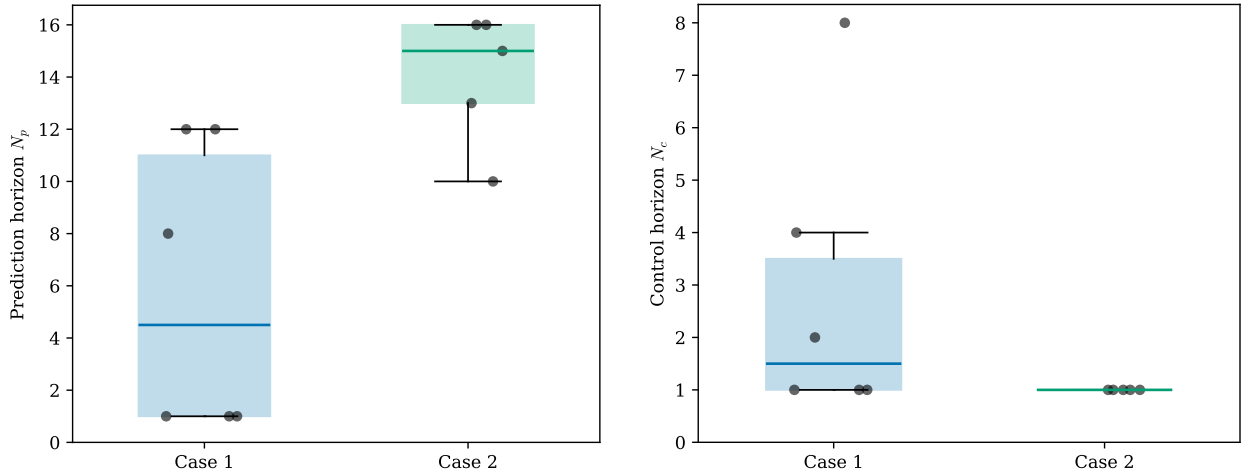


Figure 3: Objective space (J_{TV} , J_{track}) side by side, points coloured by fidelity $z = f$.

Figure 3 resolves the question of *where* the multi-fidelity mechanism concentrates effort. Low- z points spread broadly over exploratory regions of objective space, whereas high- z points cluster near the non-dominated neighbourhoods. This is the intended mechanism: cheap evaluations map global geometry, and expensive evaluations are concentrated where final Pareto quality is determined.

5.4 Horizon-selection diagnostics



(a) Prediction horizon N_p distribution.

(b) Control horizon N_c distribution.

Figure 4: Horizon distributions among Pareto-optimal candidates by run.

Figure 4 reinforces the Section 5.1 observation about horizon selection. Both runs collapse to $N_c = 1$ among Pareto candidates (9/9 in Case 1, 10/10 in Case 2). N_p is more variable: Case 2 picks larger values than Case 1 in the Pareto set, consistent with Case 2 living at the low- J_{track} end of the front (which rewards prediction-horizon length) and Case 1 living at the low- J_{TV} end (which favours the LQR-collapsed $N_c = N_p = 1$ mode).

5.5 Settling-time diagnostics

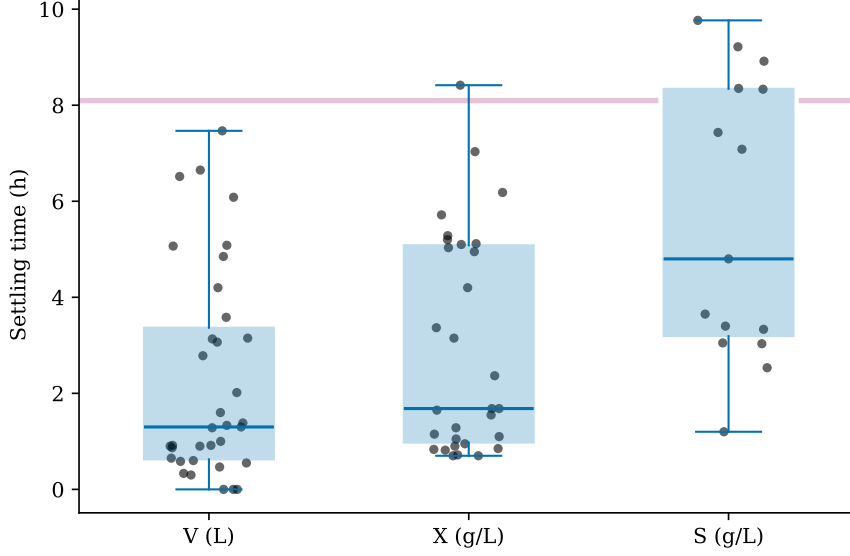


Figure 5: Settling time distributions at $z = 1$ without noise. The horizontal reference line at 8.096 03 h corresponds to $z \cdot T$ where $z = 0.81$ is the combined mean optimization-phase fidelity and $T = 10$ h is the full horizon.

Figure 5 reveals a strong state-dependent asymmetry. The fraction of runs with NaN settling times (final error above 5%) is 12.50% for state 1, 25.00% for state 2, and 62.50% for state 3. Using the 8.096 03 h threshold, settling success is 87.50% (35/40) for state 1, 72.50% (29/40) for state 2, and 25.00% (10/40) for state 3; simultaneous settling of all states is 25.00% (10/40).

The horizontal reference line at $z \cdot T = 8.096$ h carries an important secondary interpretation. State 1 mostly settles *before* this line, state 2 straddles it, and state 3 mostly does not settle. The fact that BO’s mean optimization-phase fidelity $z \approx 0.81$ coincides with the empirical settling timescale of the system is consistent with the runtime-aware acquisition having discovered an empirical truncation horizon that matches the system’s natural settling time: past that point, additional simulation samples accumulate near-zero residual error and contribute little new ranking information.

The principal control conclusion is that minimizing $(J_{\text{track}}, J_{\text{TV}})$ does not guarantee balanced state-wise settling, especially for substrate (S). This is a modelling choice in the current bi-objective formulation rather than a failure of BO itself.

5.6 Surrogate diagnostics

Figure 6 evaluates the offline Chebyshev surrogate models. The fitted degree-5 Chebyshev curves track the aggregated normalized cumulative trajectories with $R^2 = 0.9919$ for J_{TV} and $R^2 = 0.9998$ for J_{track} . The tighter fit for J_{track} suggests its cumulative growth shape is more invariant across controllers than the control-variation objective.

Figure 7 validates extrapolation on 242 processed files (121 per case) over a full-horizon reference of 10 h. Note that the horizontal axis is the normalized time within the truncated simulation,

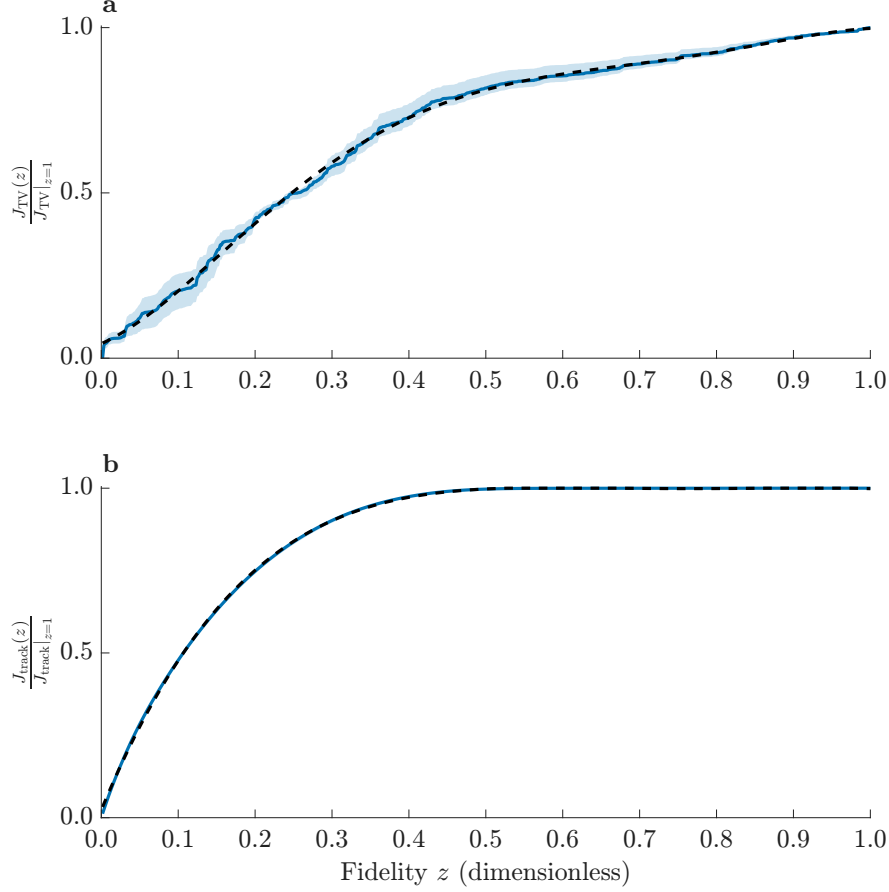


Figure 6: Aggregated surrogate-fit diagnostics: min–max envelope (shaded), empirical mean trajectory (solid), and fitted Chebyshev curve (dashed) for normalized cumulative J_{TV} and J_{track} versus fidelity z .

$\tau = t/(z \cdot T)$, so $\tau = 0.2$ corresponds to 20% of the truncated run length (i.e. to absolute time $0.2zT$) and $\tau = 1$ corresponds to the end of the truncated simulation; τ is not the fidelity itself.

The two panels behave quite differently. For J_{TV} , the median ratio approaches unity already by $\tau \approx 0.2$, which means the surrogate’s central prediction is on-average correct very early into the truncated simulation. The 95% band, however, contracts only slowly with τ : the relative-prediction variance across files remains broad even near $\tau = 1$. In practice this is the binding constraint on how low the operating fidelity z can be pushed — the surrogate is unbiased early, but the spread of extrapolation errors only narrows as the simulation runs longer. The contraction is smooth, however, so the prediction quality should not change abruptly between $z \approx 0.81$ and slightly larger or smaller fidelities. The tracking (J_{track}) surrogate, by contrast, has a much faster-contracting band but a persistent low bias: the median ratio sits below 1 across most of τ , meaning the surrogate systematically *underestimates* the true full-horizon tracking cost. Were tracking the only objective, the lower band already supports operating at fidelities 10–20% below the current operating point without a meaningful change in extrapolation quality, and the systematic bias is exactly the kind of error that online re-estimation of the Chebyshev coefficients (as discussed below) would correct.

A likely explanation for the asymmetry between the two objectives is that J_{TV} measures input motion directly, so sensitive controllers — those with aggressive weights — can produce wildly different input sequences in response to the same measurement noise, even though the plant ultimately responds similarly to those distinct sequences. The earliest samples of the truncated simulation therefore carry a much larger fraction of the controller-dependent variance for J_{TV} than they do for J_{track} , where the plant’s relatively smooth state response averages over input-level variability. Initial $J_{\text{TV}}^{\text{part}}$ estimates are consequently more sensitive to noise-driven input fluctuations than initial $J_{\text{track}}^{\text{part}}$ estimates are, which is precisely the signature visible in the 95% band of Figure 7.

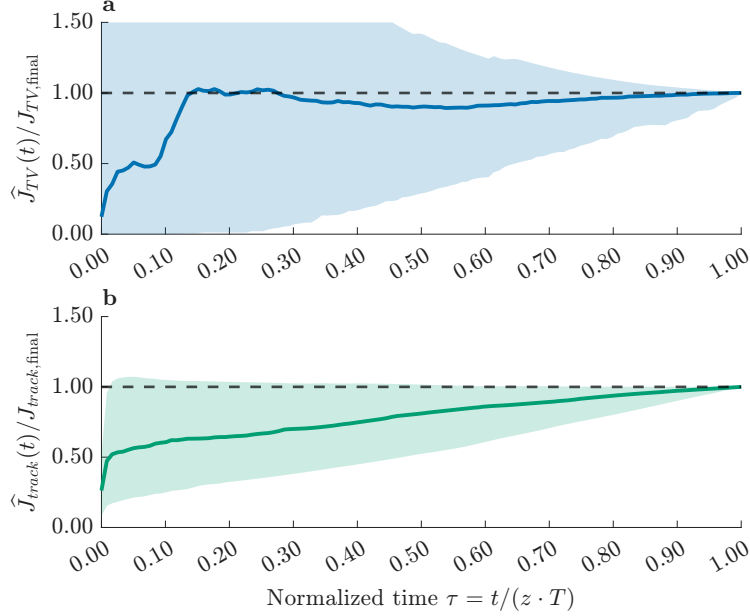


Figure 7: Pointwise 95% empirical quantile envelope for normalized extrapolated trajectories $\widehat{J}_{\text{TV}}(t)/J_{\text{TV},\text{final}}$ (top) and $\widehat{J}_{\text{track}}(t)/J_{\text{track},\text{final}}$ (bottom) across all processed outputs.

Combined endpoint absolute errors are 6.868×10^{-4} (mean) and 1.210×10^{-3} (95th percentile) for J_{TV} , and 3.568×10^{-4} (mean) and 4.418×10^{-4} (95th percentile) for J_{track} . For both objectives, 98.76% of files are within 1% endpoint error. The dominant outliers occur at very low fidelity (e.g. $z = 0.024$ with 3.353% endpoint error in J_{track}), confirming that extrapolation uncertainty is largest at extremely truncated horizons.

Model-choice observation. The surrogate $\phi(z)$ is what gets used at runtime to convert truncated cost into full-horizon cost, so its fit must be robust under online re-estimation with limited data. An unregularized degree-5 polynomial is a questionable choice for this role: six free parameters from a handful of noisy cumulative-cost trajectories invites overfitting and Runge-style oscillations near the endpoints, exactly the regime where extrapolation matters most. A more defensible choice would be a low-parameter, regularized model — a degree-2 or degree-3 monotone spline, isotonic regression, or a Beta-CDF-style family with two or three shape parameters and an L_2 penalty — and to enforce the structural constraints that the surrogate must satisfy by construction: $\phi(0) = 0$

(no cost accumulated at zero time), $\phi(1) = 1$ (normalization), and $\phi'(z) \geq 0$ on $(0, 1)$ (cumulative cost is non-decreasing). The current Chebyshev fit can violate monotonicity in principle, which is why the implementation defensively clips ϕ to $[0.01, 1]$ at evaluation time. A constrained fit removes the need for that clamp and tends to behave better under small online samples — both of which matter if the goal is to push the operating fidelity floor below the current $z \approx 0.81$.

5.7 Refined-frontier change

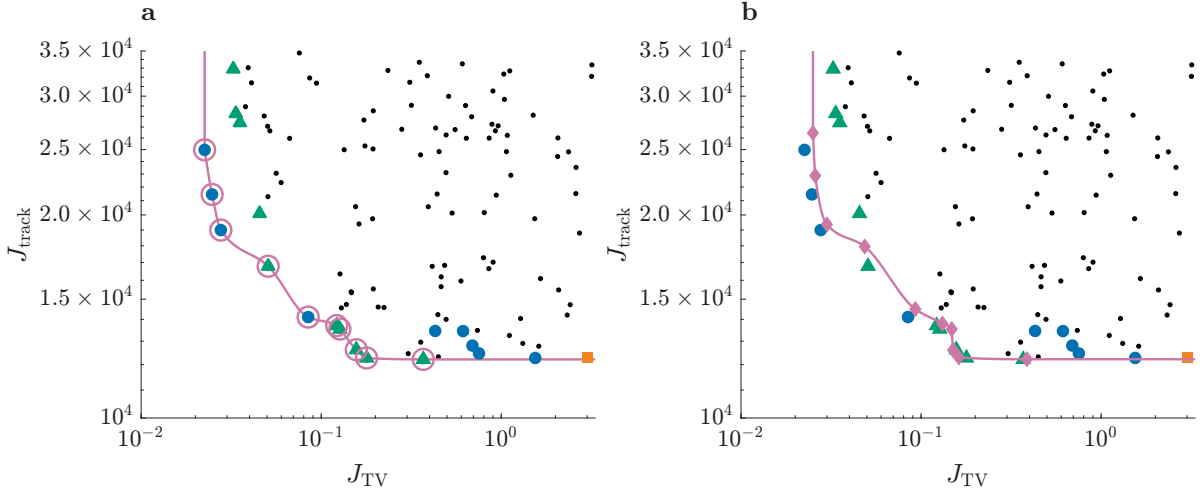


Figure 8: Combined optimization frontier (left) and original/refined comparison (right). Refined-Pareto points are highlighted as purple diamonds; controllers promoted from non-Pareto to Pareto under full-fidelity refinement (with $z < 1$ in the original) are marked as black-edged stars.

Figure 8 compares the original optimization Pareto frontier (after DOE exclusion) against the refined frontier obtained by re-evaluating each controller at $f = 1$ with the same noise. The shape of the original frontier is largely preserved, but a small number of controllers that were dominated under truncated evaluation become Pareto-optimal under full fidelity. This is precisely the mismatch the multi-fidelity assumption forces one to confront: truncated evaluation is a useful ranker, but a handful of decisions near the frontier are sensitive to the extrapolation. The list of promoted controllers is exported to `results/txt results/refined_promoted_frontier_z_lt_1.txt`.

5.8 Setpoint-schedule transferability

Figures 9 and 10 provide the strongest external-validity evidence in the project. Several modified Pareto controllers outperform the benchmark across the multi-step schedule despite the BO never having trained on this scenario: they reach each new setpoint cleanly, settle within practically usable times, and use markedly less aggressive input motion. The selected controller in Figure 9 achieves clean tracking on V and X across all three segments. Figure 10 explains *why* it outperforms the benchmark on SSdU: its F_{in} profile is smoother and less reactive than the benchmark’s, while still responding to the setpoint changes at the expected instants.

The deployment-selection workflow documented in Section 4 matters here: the BO Pareto set serves as a curated shortlist for a human decision-maker. Once the optimization narrows the field to a handful of non-dominated candidates, picking the deployment controller is a small, transparent multi-criteria choice rather than another optimization problem. The selected controller (`ts_20260211_122653_modified`) is the example of such a pick made on this codebase, motivated by simultaneous improvement in wall time, volume tracking, SSdU, and biomass IAE, against a modest substrate-tracking sacrifice.

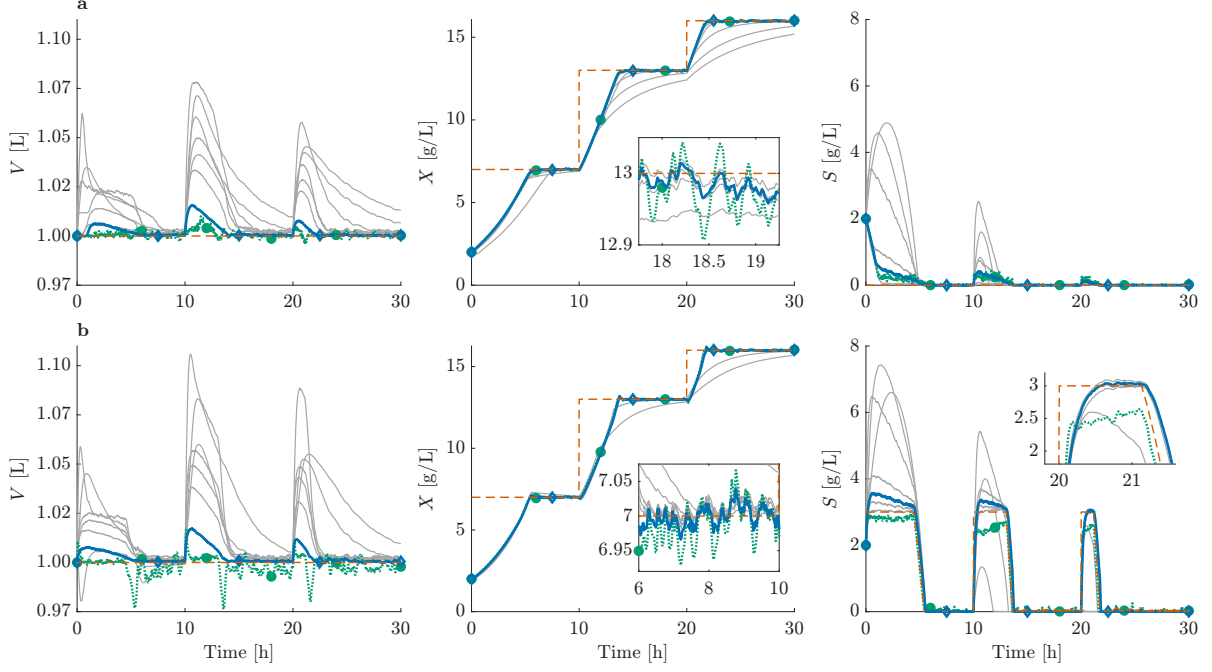


Figure 9: Validation-scenario state trajectories across both simulation-scenario initial conditions and the three schedule segments ($X_{sp} : 7 \rightarrow 13 \rightarrow 16$). The deployment-selected controller is shown as a solid blue line with diamond markers, the benchmark as a dotted vermilion line with small circular markers, and all remaining non-selected Pareto controllers as thin neutral-grey solid lines. The setpoint trajectory is drawn as a dashed reference line. Markers are placed at uniformly spaced indices to aid identification without obscuring the curves.

5.9 Input-smoothing post-analysis

Figure 11 confirms that a substantial portion of the selected controller’s high-frequency input activity can be removed without changing the cumulative input applied at each 0.1 h node. The reconstructed U^* preserves the same coarse trajectory but is visibly smoother. This suggests the BO-selected control trace contains structural slack — there is room for a tighter $R_{\Delta u}$ penalty or a post-processing smoothing step without sacrificing the tracking trajectory.

6 Discussion

Three threads run across the figures and reconcile into a single picture of how runtime-aware MFBO actually behaves on this problem.

The first concerns where the BO loop spends its budget. DOE absorbs roughly 59% of the wall time, and the optimization phase shifts to candidates that are an order of magnitude cheaper per evaluation but retain mean fidelity $z \approx 0.81$. The lower bound on z is governed by two distinct mechanisms. Physically, $z \cdot T = 8.096$ h coincides with the empirical settling timescale of the system (Section 5.5), so truncating significantly below this fidelity removes samples from a regime where many controllers are still in active transient. Statistically, the binding constraint is the J_{TV} surrogate, whose 95% extrapolation band contracts only slowly with τ (Section 5.6); the J_{track} surrogate is much more stable but biased low. Because the J_{TV} band contracts smoothly, small changes in z around the current operating point should not change extrapolation quality abruptly, and a J_{track} -only objective would already admit operating fidelities 10–20% below the present value. Online re-estimation of the surrogate (as discussed in Section 5.6) is the most direct path to reducing the bias on J_{track} and, jointly with a better-conditioned surrogate model, to pushing the effective operating fidelity lower.

The second concerns the structure of the Pareto front. Case 1 and Case 2 specialize on opposite

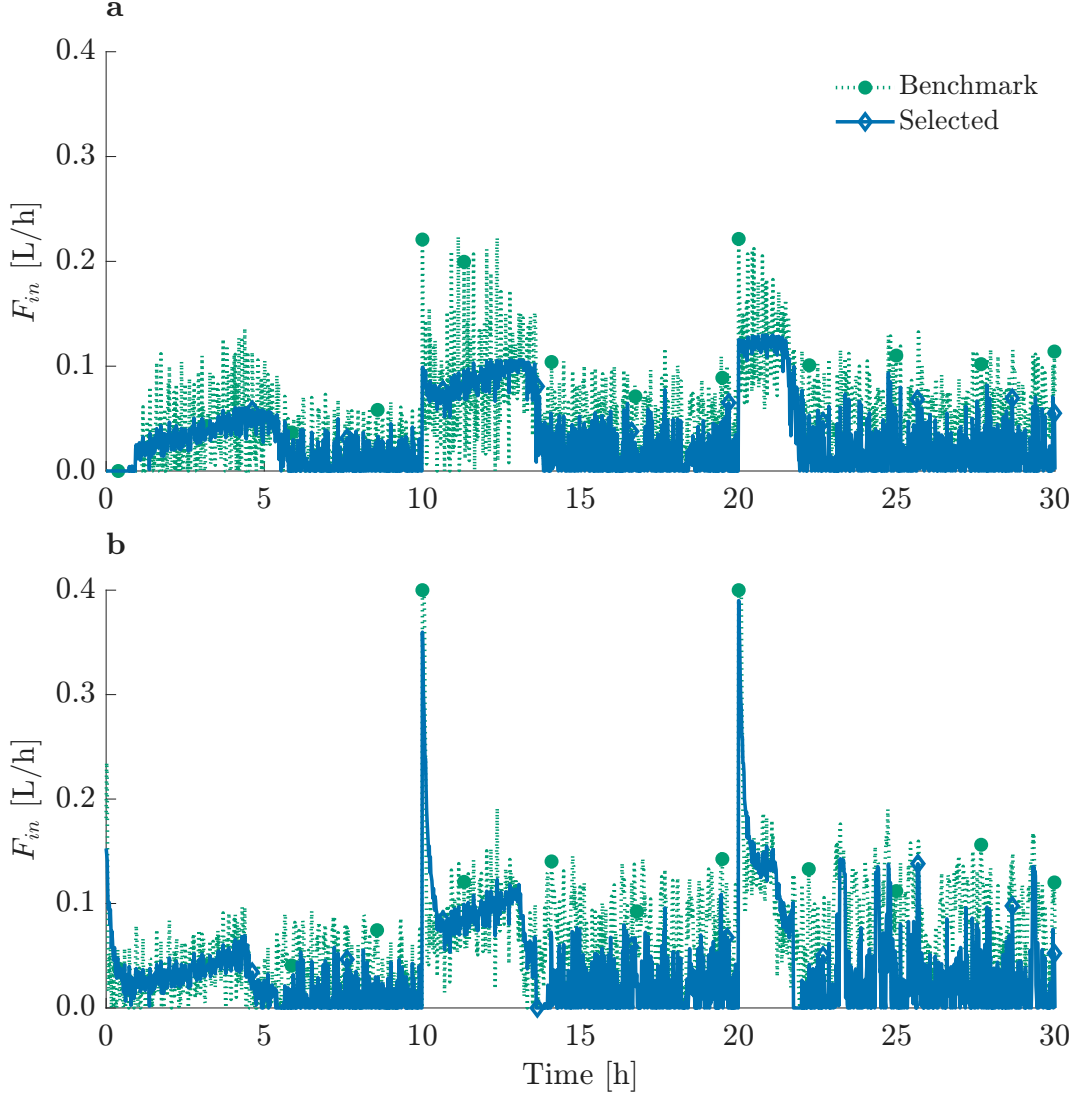


Figure 10: Manipulated-variable comparison on the validation scenario: F_{in} trajectory of the selected controller versus the benchmark, shown for both simulation-scenario initial conditions (subpanel **a**: scenario 1, subpanel **b**: scenario 2). The selected controller is plotted as a solid blue line with diamond markers placed at local maxima of its input profile, and the benchmark as a dotted vermillion line with small circular markers placed at local maxima and peak instants of its input profile. Markers are intended to highlight where each controller deviates most from a smooth setpoint-tracking response.

tails of the frontier in a way that is mechanistically tied to the NMPC horizon choices. Both runs collapse to $N_c = 1$ on the Pareto set, but Case 1 frequently also picks $N_p = 1$, which reduces the controller to the LQR feedback law derived from (6) applied to the nonlinear plant at the current state. This regime is naturally smooth (low J_{TV}) but slow (higher J_{track}). Case 2 retains $N_p > 1$, giving a single move that is anticipatory through the prediction window; the result is sharper tracking but larger move-to-move variation. The two runs are not contradictory; they are the two reachable sub-modes of an $N_c = 1$ controller architecture, and the BO finite budget made each run commit to a different one. The benchmark sits strictly inside the Pareto set, achieving comparable SSE to the best Case 2 controllers but with SSdU 8 to 22 $\times$ larger — it pays a large control-variation cost for no tracking advantage on this plant.

The third concerns deployment. Bi-objective optimization on (J_{track}, J_{TV}) does not guarantee state-wise settling, and the substrate (S) settling rate is the bottleneck (Section 5.5). The refined-frontier figure (Section 5.7) shows that a small subset of the optimization-phase Pareto rank ordering

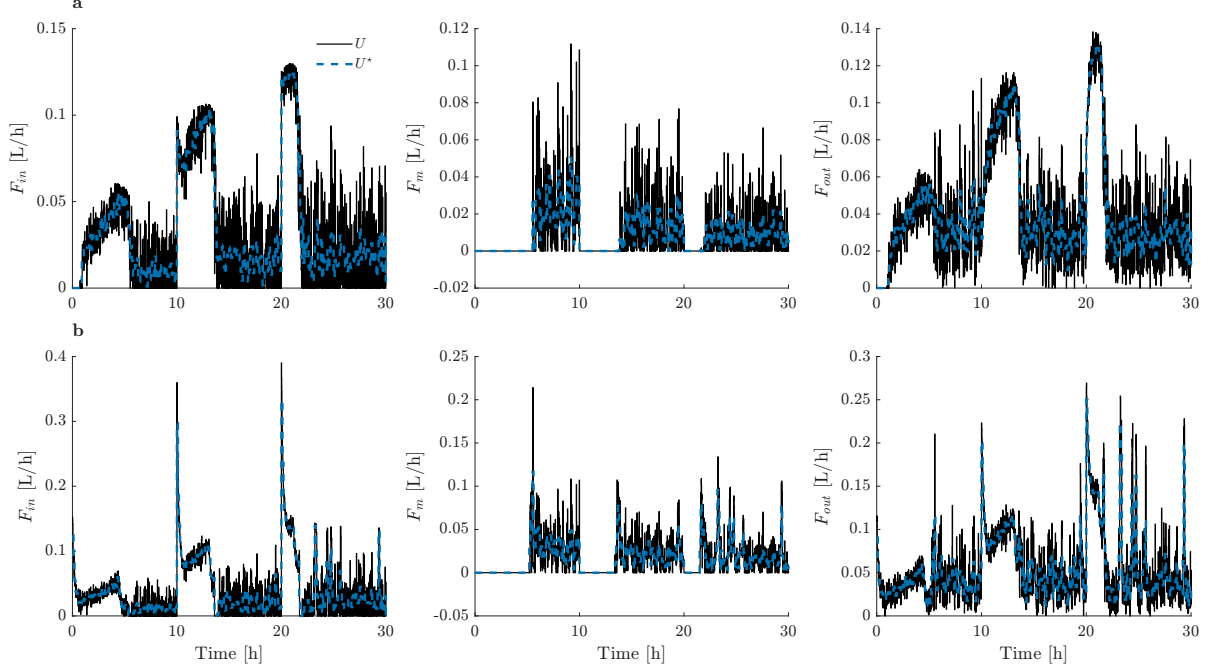


Figure 11: Original input U (black) vs reduced-TV reconstruction U^* (coloured dashed) under the cumulative-sum constraint at nodes every 0.1 h.

changes under full-fidelity replay, so the deployment recommendation should not be read directly off the BO surrogate. Instead, the workflow shown in Sections 4 and 5.8 — BO narrows the field to a small Pareto set, full-fidelity replay validates that ranking, and a human picks among a handful of candidates using a small set of practical metrics — is the correct decision pipeline. The setpoint-schedule transferability evidence is positive: several Pareto controllers beat the benchmark on a scenario the BO never saw, suggesting that the BO-Pareto-then-engineer workflow is robust to regimes outside the optimization scenario.

7 Conclusion

The `MFB0_NMPC` codebase implements a runtime-aware multi-fidelity Bayesian-optimization workflow for bi-objective NMPC tuning that operates correctly on a reduced bioprocess dilution plant. The DOE phase is the dominant cost, the optimization phase shifts to cheaper candidates while retaining high mean fidelity, and the recovered Pareto front has a robust knee that overlaps cleanly between two independent runs. The Chebyshev fraction surrogate is an excellent ranker ($R^2 = 0.992$ for J_{TV} , 0.9998 for J_{track} , 98.76% of files within 1% endpoint error), but its 95% extrapolation band is asymmetric across the two objectives: the J_{track} surrogate is biased low but tight, while the J_{TV} surrogate contracts more slowly and is the binding constraint on lowering the operating fidelity. Both observations point to the same remedy: a low-parameter, monotone, regularized surrogate that enforces $\phi(0) = 0$ and $\phi(1) = 1$, ideally re-estimated online during the BO run, would correct the J_{track} bias and tighten the J_{TV} band.

The two BO runs specialize on opposite tails of the Pareto front in a way that is mechanistically explained by the choice of N_p : the low- J_{TV} tail is the LQR-collapsed $N_c = N_p = 1$ regime, while the low- J_{track} region is the $N_c = 1, N_p > 1$ anticipatory regime. The benchmark sits strictly inside this front. The setpoint-schedule analysis demonstrates that the BO-Pareto-then-engineer workflow transfers to scenarios outside the optimization regime. State-wise settling is the principal gap: minimizing (J_{track}, J_{TV}) does not guarantee balanced settling across outputs, particularly for substrate, and a tighter settling-aware objective formulation is the recommended next iteration.